Project 3: Linear Regression

In this project, you'll work with a large data set from Cook County, Illinois (the county where Chicago is located). This data set contains over 500,000 records with 62 features, including `Sale Price.` Our ultimate goal will be to predict `Sale Price` from some combination of the other 61 features. Along the way, you will:

- Work with a large data set, including addressing issues in real-world data
- Review EDA methods, including cleaning data and adding columns
- Practice linear regression, feature engineering, and model selection

This project will have two phases. You will start with scaffolded questions, and ultimately fit and evaluate your own model.

Warning: The questions for these phases increase in difficulty as the assignment goes on.

- On phase 1 (questions 1-9), questions 7-9 all have multiple parts. You also should aim to be through question 11 by the phase 1 deadline, but only need to be through question 9.
- On phase 2, there are only 6 questions -- but the last two are the bulk of the second phase. In particular, in the last 2 questions, you will do all of: exploratory work to motivate a strong model; testing and evaluating at least five models; picking a final model that best fits the data; and building a full data science pipeline to implement that model and all the necessary feature engineering. Start early.

Partner Information

You are allowed to work with a partner on this project. If you do, only one of you should submit each phase, but that person should add their partner to the submission.

Credits

This project is adapted from Project A in the Data 100 course, with permission of the Data 100 teaching team. Please do not post or share your solutions.

In [1]:
```
import numpy as np

import pandas as pd

%matplotlib inline
import matplotlib.pyplot as plt
import seaborn as sns
```

```
import warnings
warnings.filterwarnings("ignore")

import zipfile
import os

from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
import statsmodels.api as sm

# Plot settings
plt.rcParams['figure.figsize'] = (12, 9)
plt.rcParams['font.size'] = 12
```

Phase 1: Due Wednesday 3/25 at 10pm

In phase 1, you'll explore the data, perform some EDA methods, and fit simple linear regression models.

Getting Acquainted with the Data

Again, this data set has 62 features. An explanation of each variable can be found in the included `codebook.txt` file in the project directory. Some of the rows and columns in the codebook have been filtered out to ensure this assignment doesn't become overly long when dealing with data cleaning and formatting.

The data are split into training and test sets. You have access to a training set with 163,834 observations. There is a separate test set with 33,796 observations that you do not have access to, but that you will eventually (in phase 2) use to benchmark your model.

Let's first extract the data from the `cook_county_data.zip`. Run the cells below to load the data and then verify that it is as described:

In [2]:
```
with zipfile.ZipFile('cook_county_data.zip') as item:
    with item.open("cook_county_train.csv") as f:
        initial_data = pd.read_csv(f)
```

In [3]:
```
# 163,834 observations and 62 features in training data
initial_data.shape
```

Out[3]:
```
(163834, 62)
```

In [4]:
```
initial_data.head()
```

Out[4]:

	PIN	Property Class	Neighborhood Code	Land Square Feet	Town Code	Apartments	Wall Material	Roo Materi
0	13142040110000	205	81	5250.0	71	0.0	4.0	1
1	13332260200000	203	280	3150.0	71	0.0	1.0	1
2	12293070050000	203	50	9720.0	20	0.0	1.0	1
3	16313150250000	205	70	4026.0	11	0.0	2.0	1
4	20162160280000	203	92	3250.0	72	0.0	1.0	1

5 rows × 62 columns

In [5]:
```
# Lets look at the 62 column names
# See the codebook for detailed descriptions
initial_data.columns.values
```

Out[5]:
```
array(['PIN', 'Property Class', 'Neighborhood Code', 'Land Square Feet',
       'Town Code', 'Apartments', 'Wall Material', 'Roof Material',
       'Basement', 'Basement Finish', 'Central Heating', 'Other Heating',
       'Central Air', 'Fireplaces', 'Attic Type', 'Attic Finish',
       'Design Plan', 'Cathedral Ceiling', 'Construction Quality',
       'Site Desirability', 'Garage 1 Size', 'Garage 1 Material',
       'Garage 1 Attachment', 'Garage 1 Area', 'Garage 2 Size',
       'Garage 2 Material', 'Garage 2 Attachment', 'Garage 2 Area',
       'Porch', 'Other Improvements', 'Building Square Feet',
       'Repair Condition', 'Multi Code', 'Number of Commercial Units',
       'Estimate (Land)', 'Estimate (Building)', 'Deed No.', 'Sale Price',
       'Longitude', 'Latitude', 'Census Tract',
       'Multi Property Indicator', 'Modeling Group', 'Age', 'Use',
       "O'Hare Noise", 'Floodplain', 'Road Proximity', 'Sale Year',
       'Sale Quarter', 'Sale Half-Year', 'Sale Quarter of Year',
       'Sale Month of Year', 'Sale Half of Year', 'Most Recent Sale',
       'Age Decade', 'Pure Market Filter', 'Garage Indicator',
       'Neigborhood Code (mapping)', 'Town and Neighborhood',
       'Description', 'Lot Size'], dtype=object)
```

In [6]:
```
# Let's see what the description column contains
initial_data['Description'][0]
```

Out[6]:
```
'This property, sold on 02/16/2017, is a two-story houeshold located at 4710 N SPA
ULDING AVE.It has a total of 10 rooms, 4 of which are bedrooms, and 2.0 of which a
re bathrooms.'
```

Each row in this dataset represents one property sale in Cook County and this data was collected to predict sale price values to assign property taxes. Let's familiarize ourselves with the data.

Question 1a: Put your favorite number (other than 42) into `your_fav_number` and then run the following cell. This will randomly show 3 different attributes. Look up what these mean in the code book and briefly describe each (in your own words)

In [7]:
```
# Fill the first line in and then run this cell
your_fav_number = 24
np.random.seed(your_fav_number)
np.random.choice(initial_data.columns.values, size=3, replace=False)
```

Out[7]:
```
array(['Building Square Feet', 'Age', 'Multi Property Indicator'],
      dtype=object)
```

- Building Square Feet: Building square feet, as measured from the exterior of the property
- Age: Estimated property age; defaults to 10 if missing and may reflect renovations or rebuilds.
- Multi Property Indicator: Indicator for a property with multiple improvements on one PIN,

Question 1b: Come up with at least three questions about housing in Cook County that can be answered with this dataset and provide the type of analytical tool you would use to answer it (e.g. "I would create a BLAH plot of BLAH " or "I would calculate SUMMARY STATISTIC for BLAH"). Be sure to reference the columns that you would use by their names. You can use any columns in the data set (not just the ones you looked at in 1a!)

- **Do homes with larger buildings sell for more?**
  I would create a scatterplot of `Building Square Feet` vs. `Sale Price` and fit a simple linear regression line.
- **How do sale prices vary across neighborhoods?**
  I would create grouped boxplots of `Sale Price` (or `Log Sale Price`) by `Neighborhood Code` and compute median sale price by neighborhood using a groupby summary table.
- **Are newer homes more expensive than older homes?**
  I would create a scatterplot of `Age` vs. `Sale Price` and compute the correlation between `Age` and `Log Sale Price`.
- **Do properties in floodplain areas sell for less?**
  I would create a boxplot of `Sale Price` grouped by `Floodplain` and compare group means/medians with summary statistics.

Exploratory Data Analysis

Later, you will be building a linear regression model that predicts sales prices. First, it's important to understand how the structure of the data informs such a model. In this section, we will make a series of exploratory visualizations and do some feature engineering in preparation for your final prediction task.

Sale Price

Question 2 What are the min and max sale prices of homes in `initial_data`? How many total rooms were in the most expensive house (hint: look back at the `description` column we inspected above.). Save your answer in the code cells below (you can add lines as needed). You do not need to edit any markdown cells for this question.

In [9]:
```
# BEGIN QUESTION 2
minprice = initial_data["Sale Price"].min()
maxprice = initial_data["Sale Price"].max()
max_desc = initial_data.loc[initial_data["Sale Price"].idxmax(), "Description"]
numrooms = float(max_desc.split("It has a total of ")[1].split()[0])
# END QUESTION 2
```

In [10]:
```
# Run, but do not edit, this cell
print("Min price: ", minprice)
print("Max price: ", maxprice)
print("Number rooms: ", numrooms)
try:
    if minprice <= 10000 and maxprice >= 70000000:
        print("Public test for 2 passed")
    else:
        print("Public test for 2 FAILED")
except:
    print("Public test for 2 FAILED")
```

```
Min price:  1
Max price:  71000000
Number rooms:  5.0
Public test for 2 passed
```

It seems odd that a 70+ million dollar house would have such a small number of rooms, and indeed there seems to be some sort of error. For instance, we can use the rest of the data to map that address to this Zillow page which shows a home worth decidedly less than $70 million. There seem to be some mismatches here: that house has more rooms than the description suggests, and there are no 2013 transaction details. Something seems erroneous with this entry.

Question 3 Let's look at another data point. One house had a sale price of 12 million. Use information in the data set to find that house's zillow url. Does the price history on that zillow page match what's in the `description` column for that entry?

In [40]:
```
# Scratch code for question 3
q3_house = initial_data[initial_data["Sale Price"] == 12000000]
print(q3_house[["Sale Price", "Description", "Town Code", "Town and Neighborhood"]]

q3_url = "https://www.zillow.com/homedetails/1035-Sheridan-Rd-Winnetka-IL-60093/335
print(q3_url)
```

```
       Sale Price                                        Description  \
23054    12000000  This property, sold on 05/07/2018, is a two-st...

       Town Code  Town and Neighborhood
23054         23                  23171
https://www.zillow.com/homedetails/1035-Sheridan-Rd-Winnetka-IL-60093/3359490_zpid/
```

1035 Sheridan Rd, Winnetka, IL 60093 Zillow URL:
https://www.zillow.com/homedetails/1035-Sheridan-Rd-Winnetka-IL-60093/3359490_zpid/

The Zillow sale history shows a 2018 sale around $12,000,000, so the price is broadly consistent with the dataset. However, the listing details do not fully match the description column, since Zillow lists more bedrooms and bathrooms than the dataset.

Question 4a Use seaborn to make a histogram of `Sale Price` (hint: you may want to go back to our Week 5 code demos!). Be sure to have a meaningful title. This histogram will

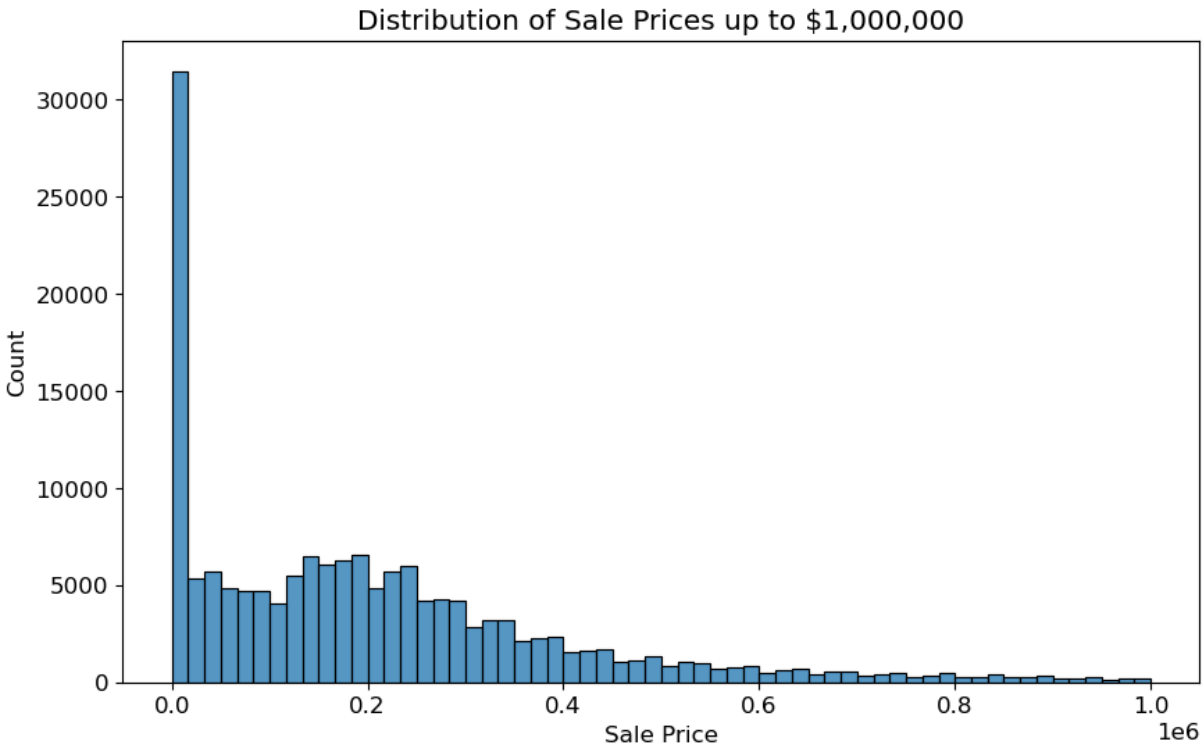
seem a bit janky because some of the sales prices are so large, as we saw before. You do not need to edit any markdown cells for this question.

In [12]:
```
# BEGIN QUESTION 4a
plt.figure(figsize=(10, 6))
sns.histplot(data=initial_data, x="Sale Price", bins=60)
plt.title("Distribution of Sale Prices in Cook County")
plt.xlabel("Sale Price")
plt.ylabel("Count")
plt.show()
# END QUESTION 4a
```



Question 4b: That histogram is hard to read. Make one that only includes sales up to $1,000,000. You do not need to edit any markdown cells for this question.

In [13]:
```
# BEGIN QUESTION 4b
plt.figure(figsize=(10, 6))
sns.histplot(data=initial_data[initial_data["Sale Price"] <= 1000000], x="Sale Pric
plt.title("Distribution of Sale Prices up to $1,000,000")
plt.xlabel("Sale Price")
plt.ylabel("Count")
plt.show()
# END QUESTION 4b
```



With this "zoomed in" histogram, we can seems to be an odd large spike near 0. Let's investigate:

In [14]:
```
initial_data['Sale Price'].value_counts()
```

Out[14]:
```
Sale Price
1         28461
250000     1009
200000      967
150000      952
180000      910
          ...
17821         1
690100        1
52111         1
249501        1
223052        1
Name: count, Length: 12697, dtype: int64
```

This should be immediately concerning. While we don't know the exact reason why so many houses were reported as $1 (perhaps these are placeholders, or transfers between family members), including these data points in any model would heavily skew the fit. After all, it's hard to imagine a house worth only $1.

Question 5 Something interesting may be happening with small-valued `Sale Price`s. In the cell below, use `value_counts()` on `Sale Prices` in the range of $0 to $499. What are the 3 most common sale prices in this range?

In [15]:
```
# BEGIN QUESTION 5
small_sale_prices = initial_data[
```

```
    (initial_data["Sale Price"] >= 0) & (initial_data["Sale Price"] <= 499)
]["Sale Price"].value_counts()
small_sale_prices.head(10)
# END QUESTION 5
```

Out[15]:
```
Sale Price
1      28461
100      216
10        15
101        5
2          1
350        1
120        1
32         1
116        1
6          1
Name: count, dtype: int64
```

The three most common sale prices between $0 and 499 are 1,100, and 10.1 is by far the most common, which suggests those very small values are not typical market sales.

Question 6 Based on the above analysis, we should pick a cutoff to get rid of smaller, probably unrepresentative `Sale Price`s. The sales price also has strong right skew, and so we will likely find it helpful to apply a log transformation (e.g. attempting to predict `log(Sale Price)` instead of `Sale Price` in our future regression models). Create a dataframe `training_data` that is identifical to `initial_data` **except with the following changes**:

1. Filter the data to only include homes that were at least $500 (inclusive) and at most $50,000,000 (inclusive). This will remove the absurdly expensive house we previously identified as erroneous as well as the small values that likely don't correspond to real home values.

2. Add a new `Log Sale Price` column that contains the log of the sale prices. Use `np.log` which calculates the natural log (i.e. log base e / $\ln$)

Then (in the next code cell) use seaborn to make a histogram of `Log Sale Price`. You do not need to edit any markdown cells for this question.

In [16]:
```
# Use this code cell to create training_data
# BEGIN QUESTION 6
training_data = initial_data[
    (initial_data["Sale Price"] >= 500) &
    (initial_data["Sale Price"] <= 50000000)
].copy()

training_data["Log Sale Price"] = np.log(training_data["Sale Price"])
# END QUESTION 6

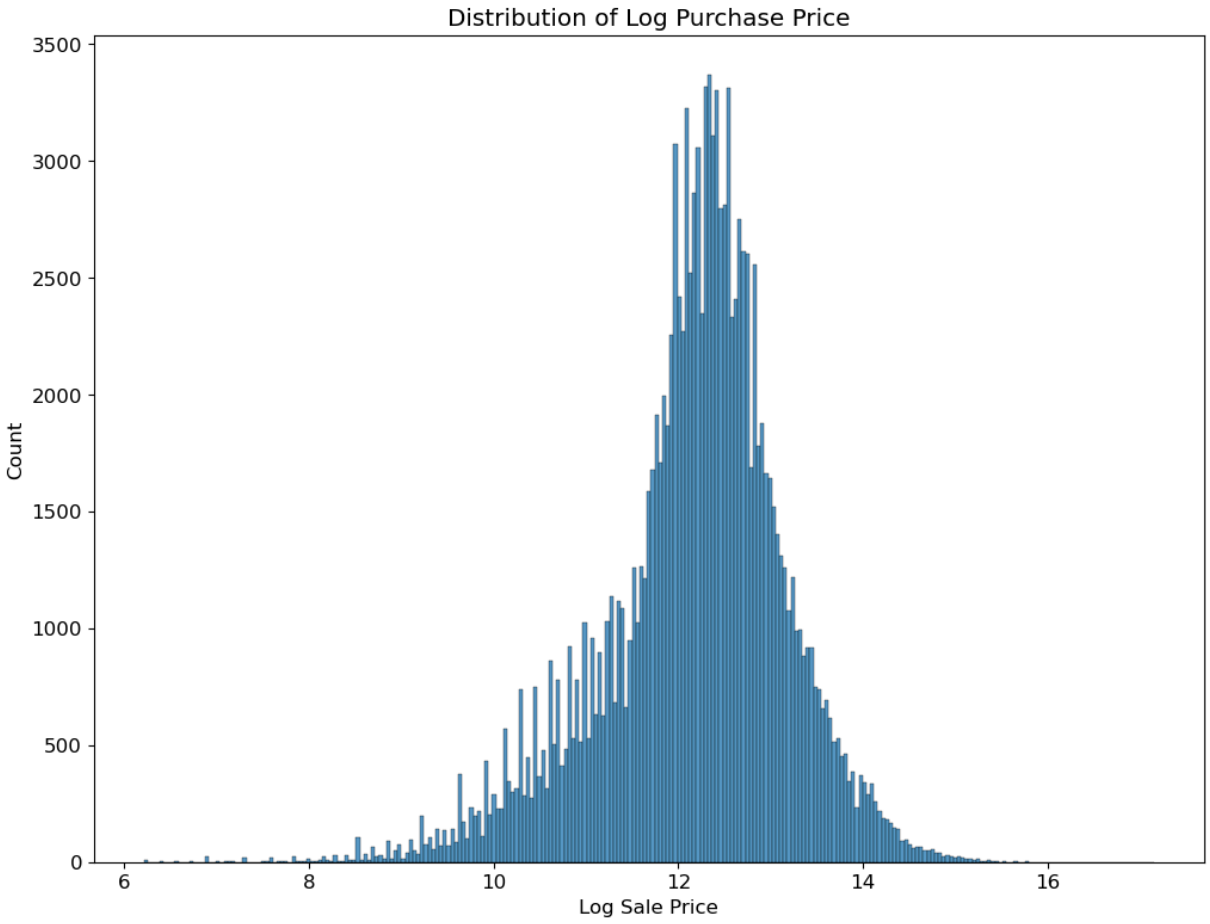

try:
```

```
    if (training_data.size == 8512812
            and 12.2<=training_data.loc[1, "Log Sale Price"]<=12.6
            and training_data["Sale Price"].max()<=50000000):
        print("Public test for 6 passed")
    else:
        print("Public test for 6 FAILED")
except:
    print("Public test for 6 FAILED")
```

```
Public test for 6 passed
```

In [17]:
```
# Use this code cell to make your histogram

sns.histplot(data=training_data, x="Log Sale Price")
plt.title("Distribution of Log Purchase Price")
plt.show()
```



We can see that `Log Sale Price` is much less skewed!

Feature Engineering: Adding and Evaluating Quantitative Predictors

One natural feature that might predict sales price is the property size. We'll use `Building Square Feet`, which measures "(from the exterior) the total area, in square feet, occupied by the building". First lets look at this feature directly

Question 7a: Use `statsmodels` to run a simple linear regression using `Building Square Feet` to predict `Log Sale Price` (including an intercept). Call your model `model_bsf`. Use the first code cell to fit your regression and obtain the summary. Then use the subsequent code cells to make a scatterplot showing both the data and your fitted line, a normal QQ plot, and a residual plot, following the guidance in the comments. Finally, use the markdown cell to summarize your work: what is the equation of the LSRL you find? How do you interpret the slope? How would you evaluate this model's fit using your diagnostic graphs and measures? As always, use polished titles/axis captions and type your math nicely (like $\hat{y}$ notation, as you've been asked to do in labs).

In [19]:
```
## Question 7a: fit your model here
X_bsf = training_data[["Building Square Feet"]].assign(intercept=1)[["intercept", "
model_bsf = sm.OLS(training_data["Log Sale Price"], X_bsf).fit()
model_bsf.summary()
```

Out[19]:

OLS Regression Results

Dep. Variable:	Log Sale Price	**R-squared:**	0.322
Model:	OLS	**Adj. R-squared:**	0.322
Method:	Least Squares	**F-statistic:**	6.428e+04
Date:	Wed, 25 Mar 2026	**Prob (F-statistic):**	0.00
Time:	21:32:30	**Log-Likelihood:**	-1.6531e+05
No. Observations:	135124	**AIC:**	3.306e+05
Df Residuals:	135122	**BIC:**	3.306e+05
Df Model:	1		
Covariance Type:	nonrobust		

	coef	std err	t	P>|t|	[0.025	0.975]
intercept	11.0846	0.005	2297.830	0.000	11.075	11.094
Building Square Feet	0.0007	2.63e-06	253.543	0.000	0.001	0.001

Omnibus:	30235.086	**Durbin-Watson:**	1.992
Prob(Omnibus):	0.000	**Jarque-Bera (JB):**	75465.709
Skew:	-1.233	**Prob(JB):**	0.00
Kurtosis:	5.706	**Cond. No.**	3.96e+03

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 3.96e+03. This might indicate that there are
strong multicollinearity or other numerical problems.

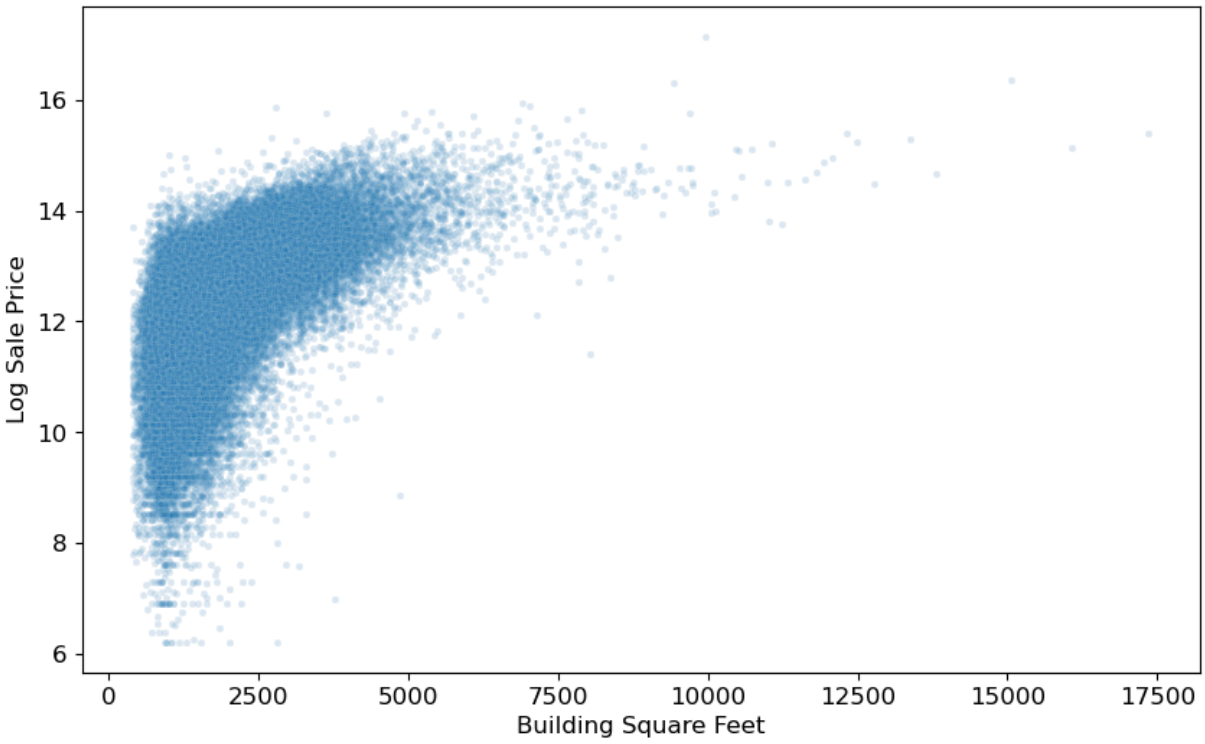
In [20]:
```
# Run, but do not edit, this cell

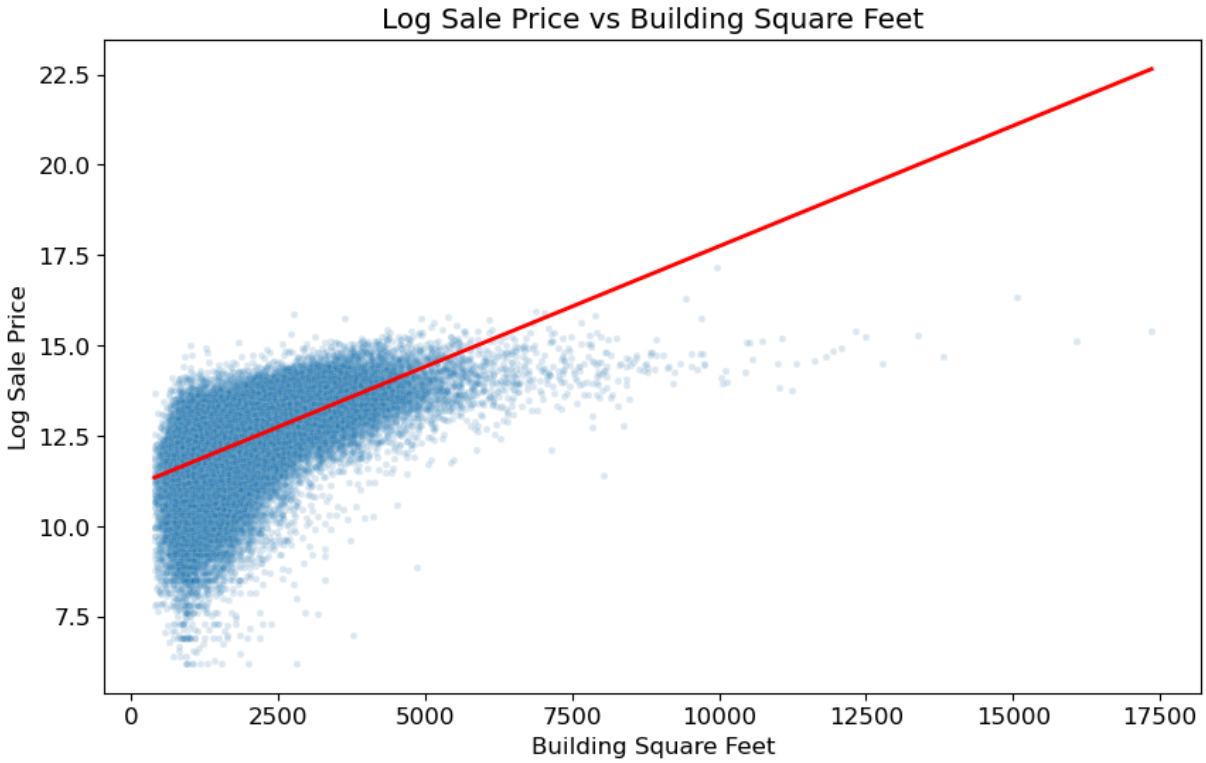
try:
    if 10 <= model_bsf.params["intercept"] <= 12 and model_bsf.params["Building Squ
        print("Public test for 7a passed")
    else:
        print("Public test for 7a FAILED")
except:
    print("Public test for 7a FAILED")
```

```
Public test for 7a passed
```

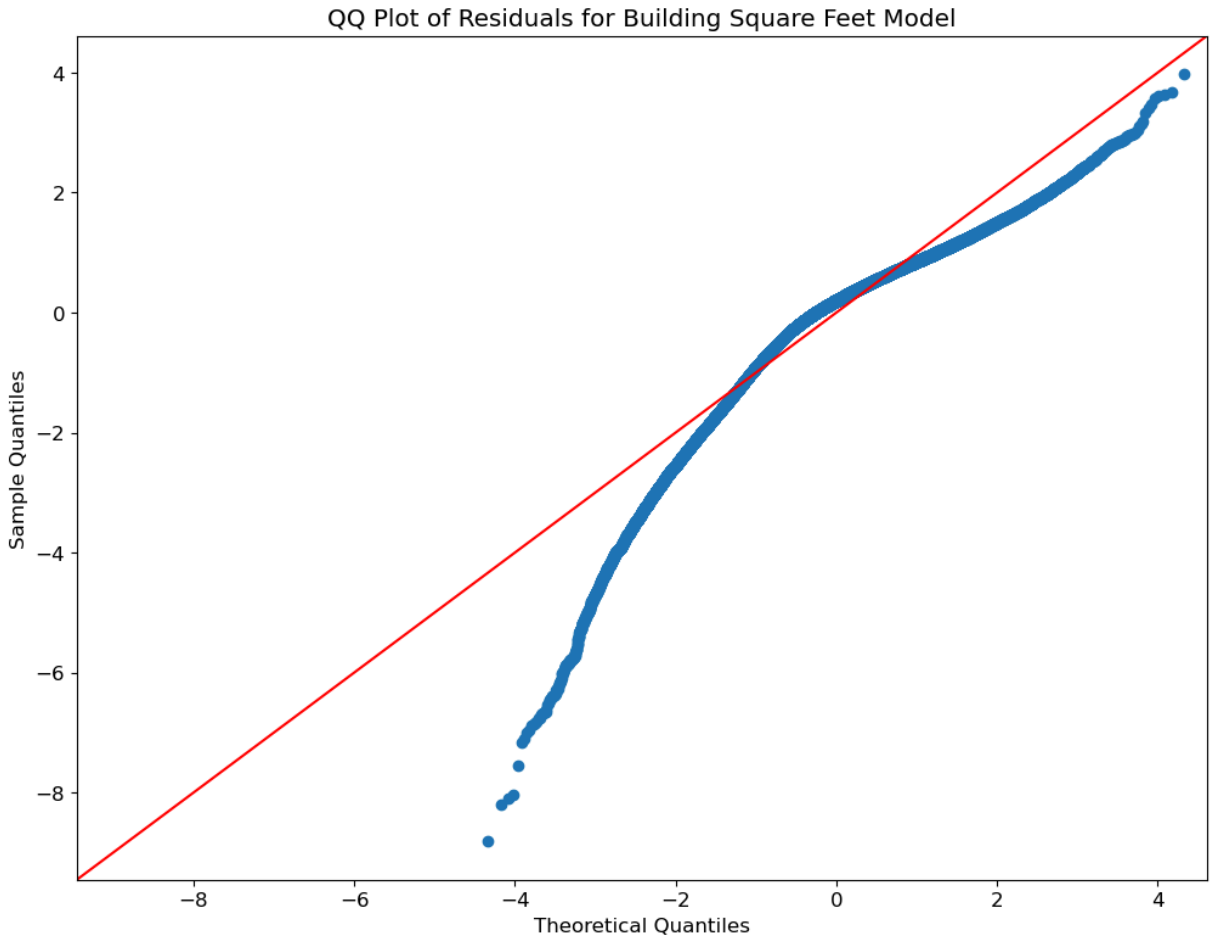
In [ ]:
```
# Question 7a: Make your scatterplot here
plt.figure(figsize=(10, 6))
sns.scatterplot(data=training_data,
                x="Building Square Feet",
```

```
                y="Log Sale Price",
                alpha=0.15,
                s=12)
x_vals = np.linspace(training_data["Building Square Feet"].min(),
                     training_data["Building Square Feet"].max(), 500)
y_vals = model_bsf.params["intercept"] + model_bsf.params["Building Square Feet"] *
plt.plot(x_vals, y_vals, color="red", linewidth=2)
plt.title("Log Sale Price vs Building Square Feet")
plt.xlabel("Building Square Feet")
plt.ylabel("Log Sale Price")
plt.show()
```

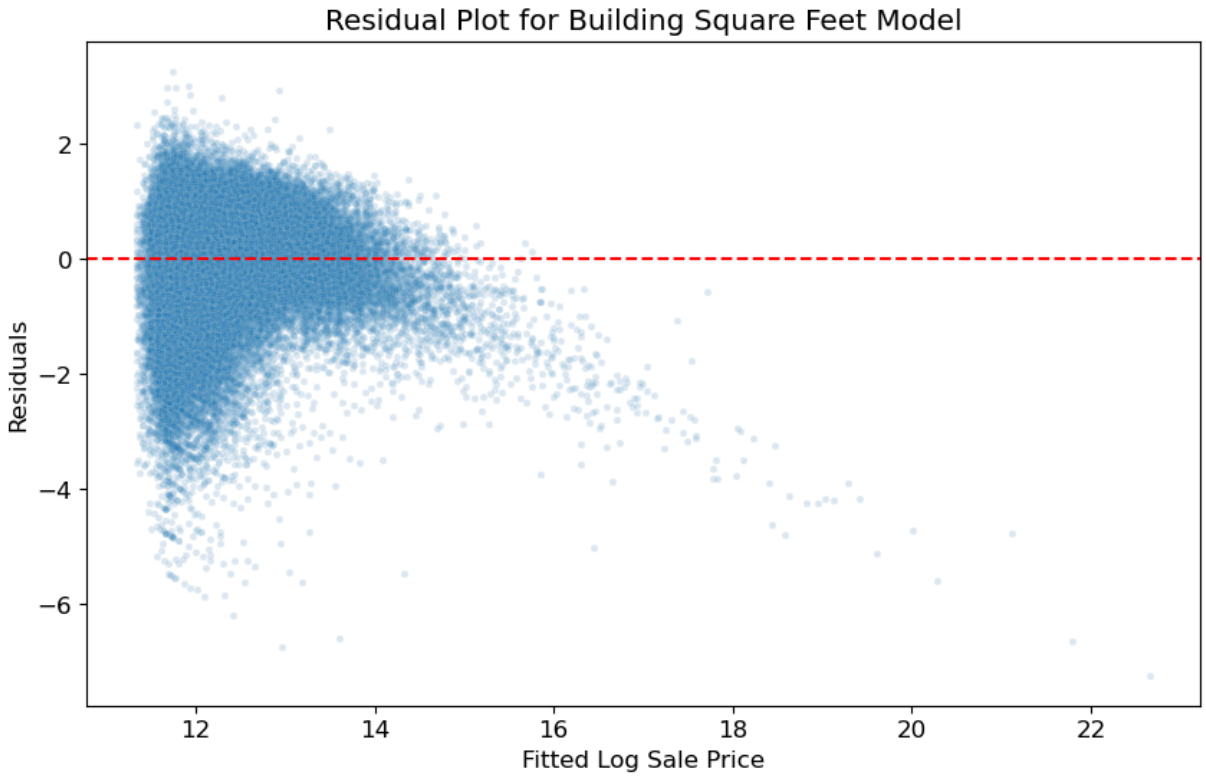




In [22]:
```
# Question 7a: Make your QQ plot here
sm.qqplot(model_bsf.resid, line="45", fit=True)
plt.title("QQ Plot of Residuals for Building Square Feet Model")
plt.show()
```



In [23]:
```
# Question 7a: Make your residual plot here
plt.figure(figsize=(10, 6))
sns.scatterplot(x=model_bsf.fittedvalues, y=model_bsf.resid, alpha=0.15, s=12)
plt.axhline(0, color="red", linestyle="--")
plt.title("Residual Plot for Building Square Feet Model")
plt.xlabel("Fitted Log Sale Price")
plt.ylabel("Residuals")
plt.show()
```



In [24]:
```
# Run, but do not edit, this cell
print("R Squared: ", model_bsf.rsquared)
```

```
R Squared:  0.3223771355200852
```

y_hat = 11.085 + 0.000666(Building Square Feet).

The slope means that each additional square foot is associated with an average increase of about 0.000666 in Log Sale Price, so an extra 100 square feet corresponds to about 0.0666 more in log price. The relationship is positive, but the fit is only moderate, with R^2 = 0.322, visible curvature in the scatterplot, some non-constant spread in the residuals, and tail departures in the QQ plot.

Question 7b: We can see in the previous scatterplot that the overall shape seems somewhat logarithmic. Let's try instead using a simple linear regression with `Log Building Square Feet` to predict `Log Sale Price` (including an intercept). For this one, create a new column `Log Building Square Feet` and then run a regression using it (and an intercept, but NOT `Building Square Feet`) to predict `Log Sale Price`.

Call this model `model_lbsf`. Use the first code cell to fit your regression and obtain the summary. Then use the subsequent code cells to make a scatterplot showing both the data and your fitted equation (the code for this one is provided for you: this should be a scatterplot of `Log Sale Price` vs `Building Square Feet` which will now be a curve), a normal QQ plot, and a residual plot. You do not need to edit any markdown cells for this question.

Note: Do NOT delete any columns from `training_data`.

In [25]:
```
## Question 7b: fit your model here
training_data["Log Building Square Feet"] = np.log(training_data["Building Square F
X_lbsf = training_data[["Log Building Square Feet"]].assign(intercept=1)[["intercep
model_lbsf = sm.OLS(training_data["Log Sale Price"], X_lbsf).fit()
model_lbsf.summary()
```

Out[25]:

OLS Regression Results

Dep. Variable:	Log Sale Price	**R-squared:**	0.348
Model:	OLS	**Adj. R-squared:**	0.348
Method:	Least Squares	**F-statistic:**	7.210e+04
Date:	Wed, 25 Mar 2026	**Prob (F-statistic):**	0.00
Time:	21:34:53	**Log-Likelihood:**	-1.6271e+05
No. Observations:	135124	**AIC:**	3.254e+05
Df Residuals:	135122	**BIC:**	3.254e+05
Df Model:	1		
Covariance Type:	nonrobust		

	coef	std err	t	P>|t|	[0.025	0.975]
intercept	2.2096	0.037	59.476	0.000	2.137	2.282
Log Building Square Feet	1.3658	0.005	268.517	0.000	1.356	1.376

Omnibus:	28289.915	**Durbin-Watson:**	1.994
Prob(Omnibus):	0.000	**Jarque-Bera (JB):**	69753.910
Skew:	-1.161	**Prob(JB):**	0.00
Kurtosis:	5.645	**Cond. No.**	126.

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

In [26]:

```
# Question 7b: Run this cell to make a scatterplot
# For this to work, you must not have deleted the Building Square Feet column from

plt.scatter(training_data['Building Square Feet'], training_data['Log Sale Price'],

x_curve =  np.linspace(training_data['Building Square Feet'].min(), training_data['

X_predict = pd.DataFrame({
    'intercept': np.ones(500),
    'Log Building Square Feet': np.log(x_curve)
})
y_fitted_line = model_lbsf.predict(X_predict)

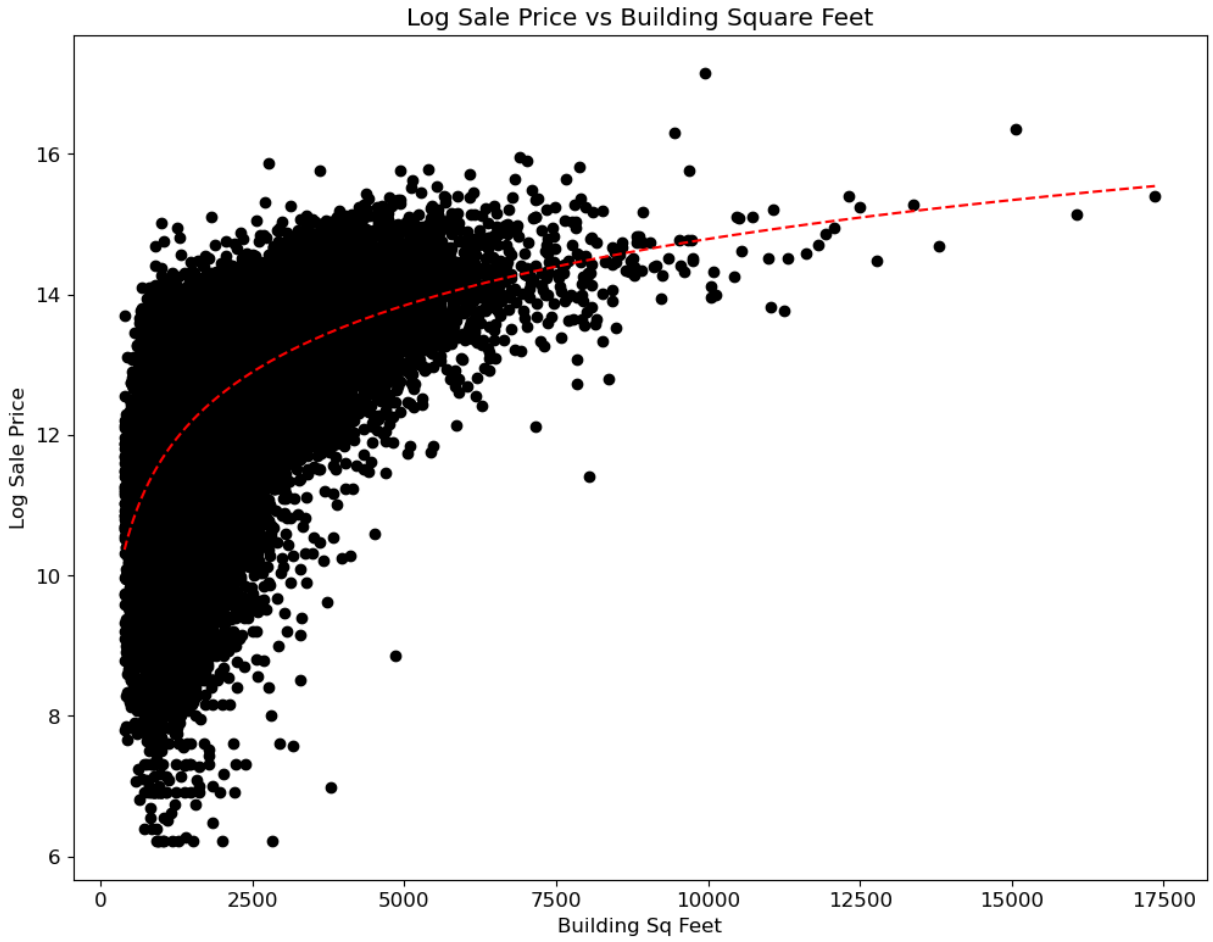

plt.plot(x_curve, y_fitted_line, color='red', linestyle='--',
         label='Fitted Line')

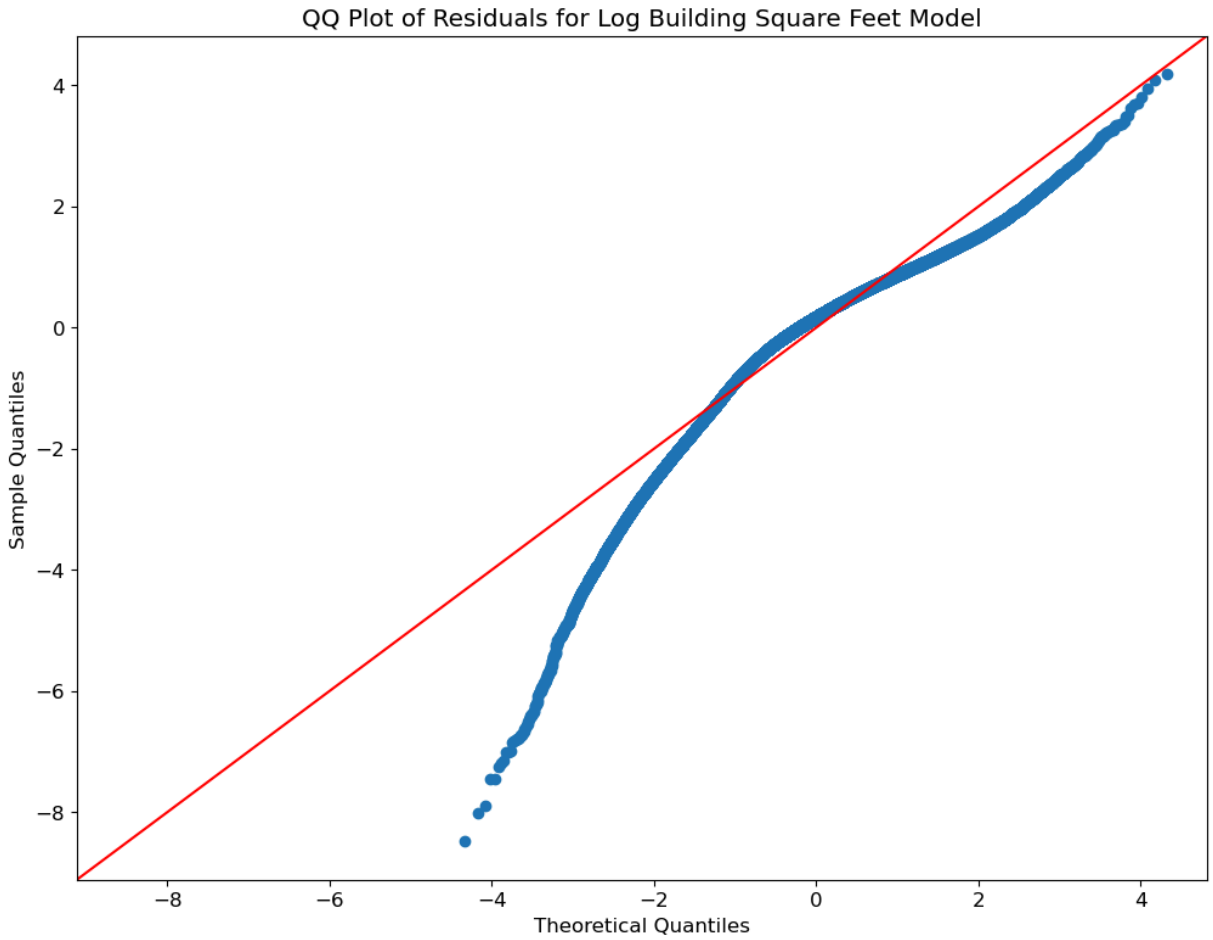
plt.title('Log Sale Price vs Building Square Feet')
```

```
plt.xlabel('Building Sq Feet')
plt.ylabel('Log Sale Price')
```

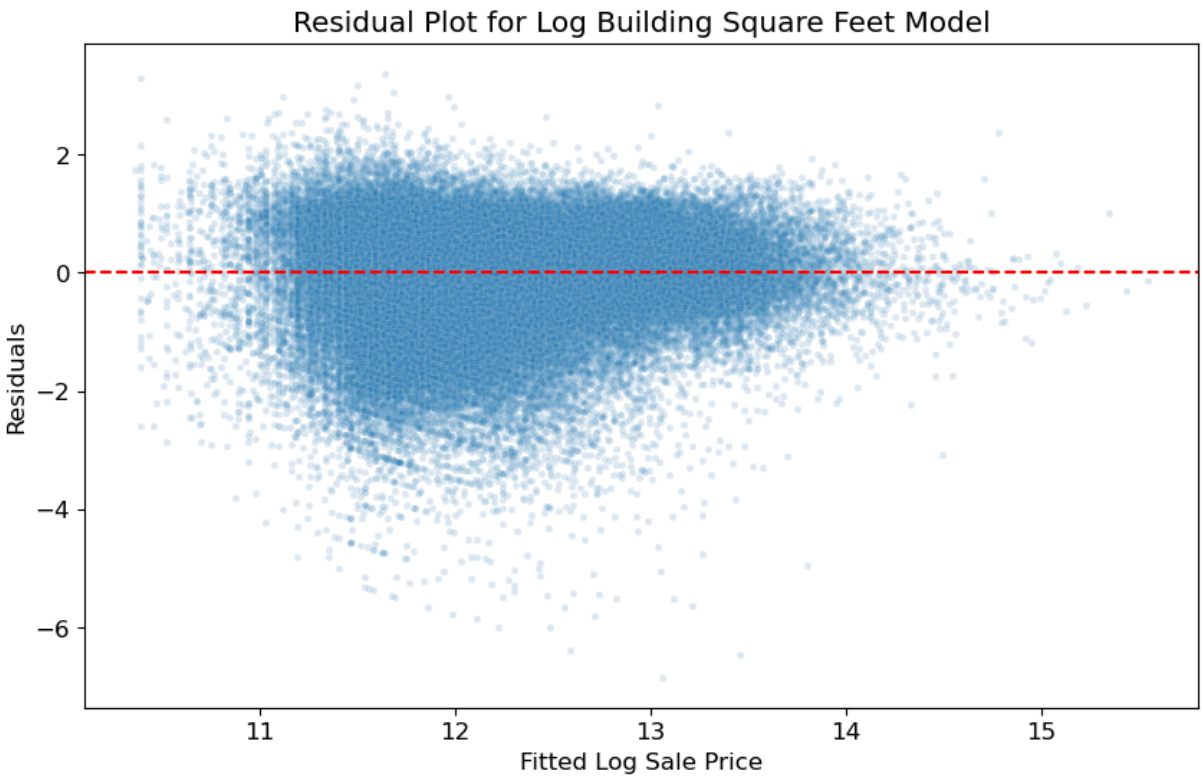
Out[26]: Text(0, 0.5, 'Log Sale Price')

In [27]:
```
# Question 7b: Make your QQ plot here
sm.qqplot(model_lbsf.resid, line="45", fit=True)
plt.title("QQ Plot of Residuals for Log Building Square Feet Model")
plt.show()
```





In [28]:
```
# Question 7b: Make your residual plot here
plt.figure(figsize=(10, 6))
sns.scatterplot(x=model_lbsf.fittedvalues, y=model_lbsf.resid, alpha=0.15, s=12)
plt.axhline(0, color="red", linestyle="--")
plt.title("Residual Plot for Log Building Square Feet Model")
plt.xlabel("Fitted Log Sale Price")
plt.ylabel("Residuals")
plt.show()
```



Question 7c: Based on the above, which do you think will be a better predictor of `Log Sale Price`? Explain your reasoning, drawing off of the above work.

Log Building Square Feet is the better predictor because it makes the relationship more linear, slightly improves the fit (R^2 rises from about 0.322 to about 0.348), and gives cleaner residual patterns.

Another natural predictor might be the number of rooms/bedrooms/bathrooms. Those features are, however, conspicuously absent from our columns. Instead, we've seen them in the description attribute. For instance:

In [29]:
```
initial_data.loc[1, "Description"]
```

Out[29]:
```
'This property, sold on 07/01/2016, is a one-story with partially livable attics h
oueshold located at 2048 N LAVERGNE AVE.It has a total of 5 rooms, 3 of which are
bedrooms, and 1.5 of which are bathrooms.'
```

Question 8a Write a function `split_desc` that will take in a description (like the above, as a string) and use string split methods to return a list of the form `[number rooms, number bedrooms, number bathrooms]`. Each should be a float. Hint: the string `split` method will help! You do not need to edit any markdown cells for this question.

In [34]:
```
# BEGIN QUESTION 8a
def split_desc(description):
    details = description.split("It has a total of ")[1]
    pieces = details.split(", ")

    rooms = float(pieces[0].split()[0])
```

```
    bedrms = float(pieces[1].split()[0])
    baths = float(pieces[2].split()[1])

    return [rooms, bedrms, baths]
# END QUESTION 8a




try:
    print(split_desc(initial_data.loc[42, "Description"]))
    [a, b, c] = split_desc(initial_data.loc[1, "Description"])
    if a==5.0 and b==3.0 and c==1.5 and type(b)==type(a)==type(c)==float:
        print("Public test for 8a passed")
    else:
        print("Public test for 8a FAILED")
except:
    print("Public test for 8a FAILED")
```

```
[5.0, 3.0, 2.0]
Public test for 8a passed
```

Question 8b Apply your function `split_desc` to create three new columns in `training_data`: `num_room`, `num_bedrm`, and `num_bath`, respectively storing the total number of rooms, total number of bedrooms, and total number of bathrooms in each house. Hint: you may want to create simple helper functions (like `get_rooms`) that get one attribute at a time (these can be one liners, since you already wrote `split_desc`). Then use the `apply` pandas method to create your columns one at a time. You do not need to edit any markdown cells for this question.

In [35]:
```
# BEGIN QUESTION 8b
def get_rooms(description):
    return split_desc(description)[0]

def get_bedrooms(description):
    return split_desc(description)[1]

def get_bathrooms(description):
    return split_desc(description)[2]

training_data["num_room"] = training_data["Description"].apply(get_rooms)
training_data["num_bedrm"] = training_data["Description"].apply(get_bedrooms)
training_data["num_bath"] = training_data["Description"].apply(get_bathrooms)
# END QUESTION 8b



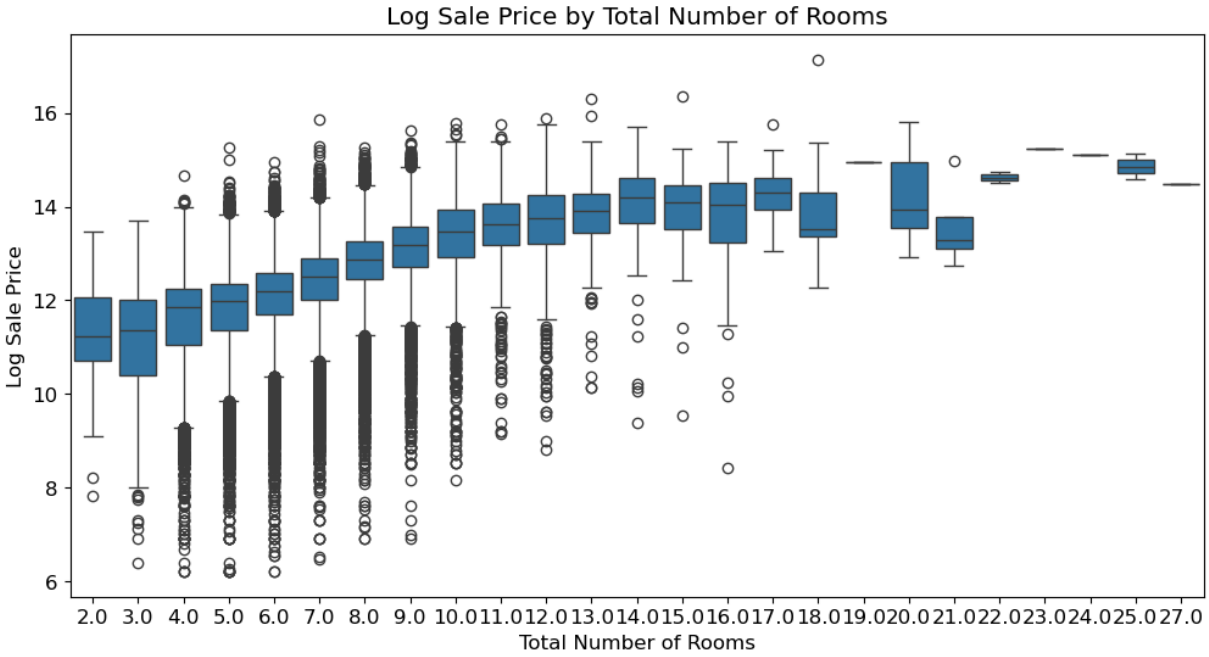
training_data.head()
try:
    a = training_data.loc[1, "num_room"]
    b = training_data.loc[1, "num_bedrm"]
    c = training_data.loc[1, "num_bath"]
    if a==5.0 and b==3.0 and c==1.5:
```

```
        print("Public test for 8b passed")
    else:
        print("Public test for 8b FAILED")
except:
    print("Public test for 8b FAILED")
```

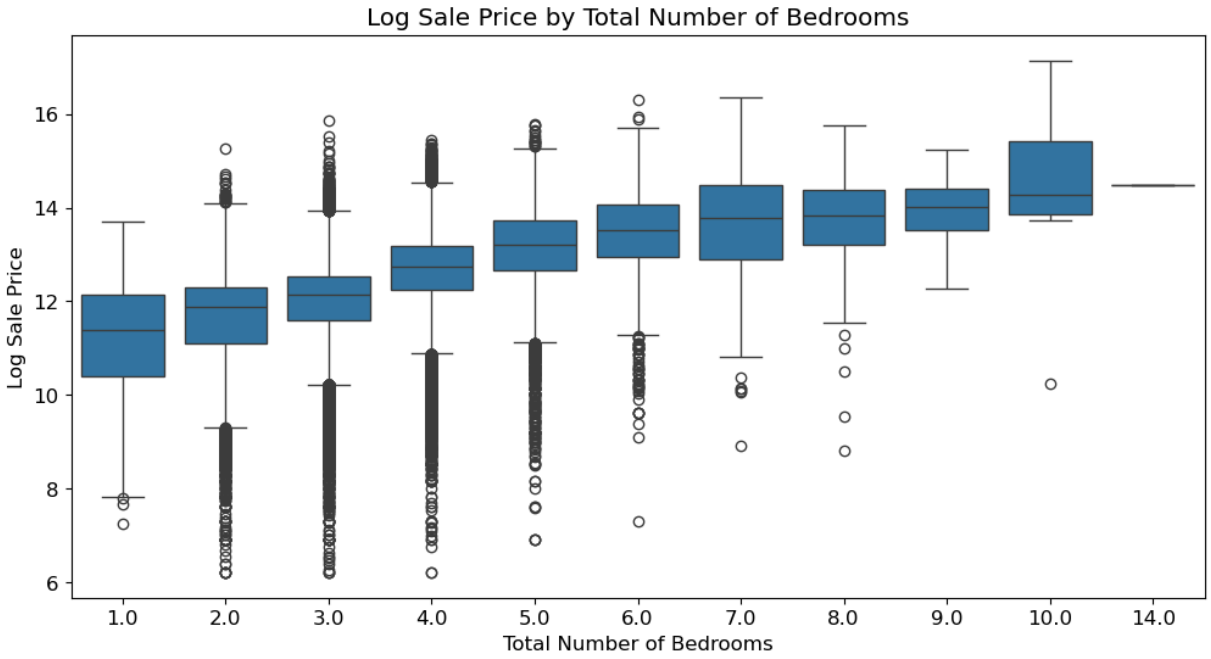
```
Public test for 8b passed
```

Question 8c Use Seaborn to make 3 clustered boxplots. In all three, the y axis should be `Log Sale Price.` In one, you should cluster by the number of total rooms, in one by the total number of bedrooms, and in one by the total number of bathrooms. Don't forget axis labels and a title, and the Week 5 code demos will help you out :). You do not need to edit any markdown cells for this question.

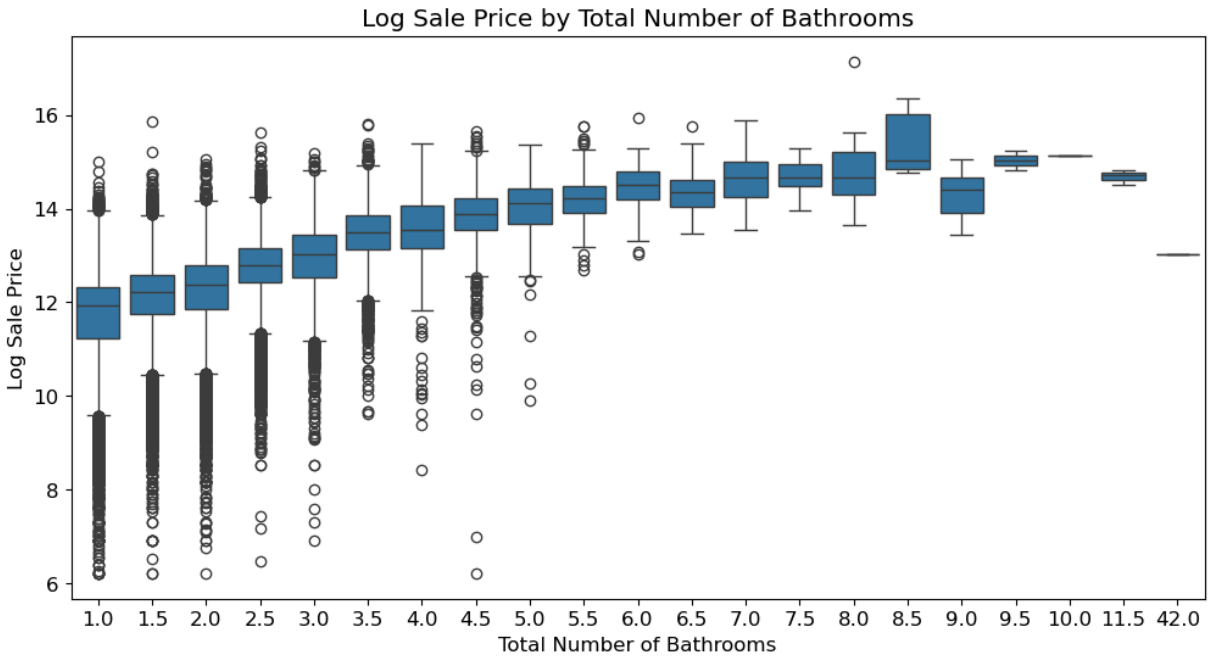
In [36]:
```
# Group Log Sale Price by Total Number of Rooms Here
plt.figure(figsize=(12, 6))
sns.boxplot(data=training_data, x="num_room", y="Log Sale Price")
plt.title("Log Sale Price by Total Number of Rooms")
plt.xlabel("Total Number of Rooms")
plt.ylabel("Log Sale Price")
plt.show()
```



In [37]:
```
# Group Log Sale Price by Total Number of Bedrooms Here
plt.figure(figsize=(12, 6))
sns.boxplot(data=training_data, x="num_bedrm", y="Log Sale Price")
plt.title("Log Sale Price by Total Number of Bedrooms")
plt.xlabel("Total Number of Bedrooms")
plt.ylabel("Log Sale Price")
plt.show()
```



In [38]:
```
# Group Log Sale Price by Total Number of Bathrooms Here
plt.figure(figsize=(12, 6))
sns.boxplot(data=training_data, x="num_bath", y="Log Sale Price")
plt.title("Log Sale Price by Total Number of Bathrooms")
plt.xlabel("Total Number of Bathrooms")
plt.ylabel("Log Sale Price")
plt.show()
```



Question 9a Which of number of rooms, number of bedrooms, and number of bathrooms do you think is the most predictive of Log Sale Price? Explain your reasoning in 1 sentence.

The number of bathrooms looks most predictive because its boxplots show the clearest upward trend in Log Sale Price as bathroom count increases.

Question 9b You should notice one clear and extreme outlier in your grouped boxplots. Figure out what house it corresponds to and find the zillow listing. Specifically, give the index number (that would allow you to access it using `.loc`) and the zillow URL for it. Verify that the zillow URL shows a sale matching the sale price around the date of sale. Based on the listing, is there an error in the data?

In [ ]:
```
# BEGIN QUESTION 9b
outlier = training_data[(training_data["num_bath"] == 1.5)
                        & (training_data["Sale Price"] > 5000000)]
outlier[["Sale Price", "Description", "num_room", "num_bedrm", "num_bath"]]

outlier_url = "https://www.zillow.com/homedetails/1029-Prairie-Ave-Park-Ridge-IL-60
print(outlier_url)
# END QUESTION 9b
```

```
https://www.zillow.com/homedetails/1029-Prairie-Ave-Park-Ridge-IL-60068/3507863_zpi
d/
```

The extreme outlier is index 40329, corresponding to 1029 Prairie Ave, Park Ridge, IL 60068.

Zillow URL: https://www.zillow.com/homedetails/1029-Prairie-Ave-Park-Ridge-IL-60068/3507863_zpid/

Zillow's sale history shows a sale around October 2015 for about $780,000, not 7,800,000$, so this appears to be a data error, likely an extra zero.

You have now made it through phase 1! You are welcome to continue working on the questions below (and again, being through question 11 by the phase 1 deadline is a good target!), but you only have to have the above completed for phase 1!

Phase 1 Submission Instructions

For Phase 1, you will submit to **two** separate Gradescope assignments: a PDF and your Jupyter notebook. This will work just like in Project 2

1. Restart the Kernel (e.g. Kernel --> Restart) and run all cells above this one (e.g. Cell --> Run All Above).
2. Make sure that all of the above cells run without error, including the cells that say `# Run, but do not edit, this cell`. Be sure that you pass the provided tests.
3. Create a PDF of your notebook. To do so, (1) hit File -> Download as -> HTML (or potentially Save and File -> Export Notebook As -> HTML, depending on your version of Jupyter/OS) (2) open that HTML file in Google Chrome (I drag from downloads into a new Chrome tab), and (3) save that HTML file as a PDF (I type `ctrl+P` to print and then select `Save as PDF` as my destination).

4. Skim the PDF and make sure all your code and written responses are fully visible. If you notice anything (like your code lines are too long and get cut off; your written answers are cut off), fix those issues and repeat the above process. If I cannot see your work, it will not be graded.
5. Upload that PDF to `Project 3 Phase 1: PDF Submission` on Gradescope. Tag all pages with the relevant question, and if you worked with a partner, add them to your submission.
6. Upload your Project 3 Jupyter notebook to `Project 3 Phase 1: Jupyter Submission` on Gradescope. If you worked with a partner, add them to your submission. Look at the code you uploaded and verify that you submitted a complete Jupyter notebook, with answers to the above questions / cells run. You do not (and should not) zip anything, include data sources, etc -- just upload your .ipynb file.

As always, if you run into any issues, please be sure to send me documentation of your completed work (e.g. a completed Jupyter notebook) before the deadline.

Phase 1 Grading Information.

Meeting the Phase 1 checkpoint is worth 15 points of your submission. You will get 2 automatic autograder points for submitting your Jupyter notebook (note that the autograder is only checking that you've included a Jupyter notebook file -- you should independently verify that it actually has the right input and output). This submission is a backup so that I can see what code was completed on time.

This phase will primarily be graded on completeness: I will not carefully check your code or your written answers, but will verify that you have responded to all questions with reasonably polished answers, that you have passed the provided tests, and that you have written appropriate code. Your work will be checked on completeness with your phase 2 submission -- so if you discover that you made errors in the above, you are welcome to correct them for your future submissions.

Phase 2: Due Wednesday 4/1 at 10pm

In this phase, you'll do a bit more feature engineering, build a data pipeline, and select your best possible model! If you're picking this back up, be sure to rerun the above cells.

Feature Engineering - Categorical Variables

So far, we've done feature engineering with quantitative variables. Lets consider some of our categorical variables. For instance, more expensive neighborhoods may command a

In [ ]:
```
len(training_data["Neighborhood Code"].unique())
```

We could try making a boxplot to look at log sale price by neighborhood code, but the number of neighborhoods make this practically useless:

In [ ]:
```
sns.boxplot(data=training_data, x='Neighborhood Code', y='Log Sale Price')
```

We could try to look at a subset (say, just the most popular). Instead, lets try to identify the most expensive neighborhoods by median sale price.

Question 10a: Complete the following function `find_expensive_neighborhoods` which should take in a dataframe `df` (including a column `Sale Price`) and an input `n`. It should return a list of the `Neighborhood Code`'s of the top `n` most expensive neighborhoods, as measured by their median `Sale Price`. Hint: This sounds like a groupby!! You do not need to edit any markdown cells for this question.

In [ ]:
```
# BEGIN QUESTION 10a

def find_expensive_neighborhoods(df, n=3):
    """
    Input:
      df (DataFrame): should contain at least an int-valued 'Neighborhood Code'
        and a numeric 'Sale Price' column
      n (int): the number of top values desired

    Output:
      a list of the the neighborhood codes of the top n highest-priced neighborhood
      as measured by the median Sale Price of each
    """

    neighborhoods = ... # Maybe use a groupby here (and some other ops, possibly ov

    # This makes sure the final list contains the generic int type used in Python3,
    return [int(code) for code in neighborhoods]


# END QUESTION 10a

expensive_neighborhoods = find_expensive_neighborhoods(training_data, 3)


try:

    expensive_neighborhoods = find_expensive_neighborhoods(training_data, 3)
    if len(expensive_neighborhoods)==3 and 44 in expensive_neighborhoods:
        print("Public test for 10a passed")
    else:
        print("Public test for 10a FAILED")
except:
    print("Public test for 10a FAILED")
```

```
expensive_neighborhoods
```

Question 10b: Use `find_expensive_neighborhoods` to add a column called `in_expensive` to `training_df` that is 1 if a house is in one of the 10 most expensive neighborhoods (by median Sale Price) and 0 otherwise. Hint: remember `np.where`. You do not need to edit any markdown cells for this question.

In [ ]:
```
# BEGIN QUESTION 10b


# END QUESTION 10b

try:
    if training_data.loc[445, "in_expensive"]==1 and training_data.loc[0, "in_expen
        print("Public test for 10b passed")
    else:
        print("Public test for 10b FAILED")
except:
    print("Public test for 10b FAILED")

print(training_data["in_expensive"].sum())
```

Question 10c: Make a boxplot where you group `Log Sale Price` by `in_expensive`. We should expect to see a reasonably strong association since we've explicitly defined `in_expensive` to relate to neighborhoods with higher sale prices. You do not need to edit any markdown cells for this question, but should (as always in a project!) have good titles and axis labels

In [ ]:
```
# BEGIN QUESTION 10c
# END QUESTION 10c
```

Now lets turn our attention to `Wall Material` which has the following levels:

In [ ]:
```
training_data["Wall Material"].value_counts()
```

And does have some relationship with price:

In [ ]:
```
sns.boxplot(data=training_data, x='Wall Material', y='Log Sale Price')
```

Question 11a Explain precisely why it does not make sense to treat `Wall Material` as a quantitative variable in our model, and what would go wrong if we did so. Hint: look at the codebook to understand what wall material stores. Think about the difference between a wood and and masonry wall, versus a stucco and wood&masonry wall.

Edit this cell with your answer to 11a

Question 11b: Use the `replace` method for DataFrames to create a new column, `Wall Material Cat`, which renames the levels of `Wall Material` to their proper names

(Wood, Masonry, Wood&Masonry, and Stucco). You should not change the column `Wall Material`. You do not need to edit any markdown cells for this question.

In [ ]:
```
# BEGIN QUESTION 11b

# END QUESTION 11b


try:
    n1 =  len(training_data[training_data["Wall Material"]==3])
    n2 =  len(training_data[training_data["Wall Material Cat"]=='Wood&Masonry'])
    if n1==n2:
        print("Public test for 11b passed")
    else:
        print("Public test for 11b FAILED")
except:
    print("Public test for 11b FAILED")


print(training_data["Wall Material Cat"].value_counts())
```

Question 11c Finally, use the `get_dummies` method to create new columns based on `Wall Material Cat`. Use `drop_first=True` so that this create 3 new columns. You can do this in one line. You do not need to edit any markdown cells for this question.

Note: You may run into issues if you run this cell multiple times.

In [ ]:
```
# BEGIN QUESTION 11c


# END QUESTION 11c

try:
    n1 =  len(training_data[training_data["Wall Material Cat"]=='Wood&Masonry'])
    n2 = training_data["Wall Material Cat_Wood&Masonry"].sum()
    if n1==n2:
        print("Public test for 11c passed")
    else:
        print("Public test for 11c FAILED")
except:
    print("Public test for 11c FAILED")
```

In [ ]:
```
# Run, but do not edit, this cell

training_data[["PIN", "Wall Material",  "Wall Material Cat_Stucco", "Wall Material
```

OK! We've now seen how to:

- Tranform quantitative variables and use scatterplots to guide our intuition for those transformations (as with sale price and building size)
- Extract variables from text (as with number rooms/bedrooms/bathrooms)

- Create a categorical variable summarizing a variable with many categories (as with Neighborhood ID)
- Take a categorical column coded as a quantitative column, rename it, and split it into dummy columns.

That completes our jaunt through several feature engineering methods. Let's put it together!

Question 12a Use statsmodels to fit a simple linear regression model `mod1` where you use `Building Square Feet` (and an intercept) to predict `Log Sale Price`. Hint: You did this in 7a, so you can likely just copy and paste your work and change the name to now be `mod1`. You do not need to edit any markdown cells for this question, and you do not need to make any diagnostic plots.

In [ ]:
```
## Question 12a: fit your model here
```

In [ ]:
```
# Run, but do not edit, this cell
mod1.summary()
```

Question 12b Use statsmodels to fit a linear regression model `mod2` where you try to predict `Log Sale Price` using:

- Log Building Square Feet
- num_bedrm
- in_expensive
- your 3 dummy variables from 11c
- (and an intercept).

You do not need to edit any markdown cells for this question, and you do not need to make any diagnostic plots.

In [ ]:
```
## Question 12b: fit your model here
```

In [ ]:
```
# Run, but do not edit, this cell
mod2.summary()
```

Question 12c How do the R^2 compare for your model? How do you interpret the coefficient of `num_bedrm` and how do you explain the fact that it's negative and statistically significant? How do you interpret the coefficient of `Wall Material Cat_Stucco`?

Edit this cell with your answer to 12c

Run the following cells to compare the MSE of your models on the training data. We should see the MSE drop slightly (remember that lower MSE = better!).

In [ ]:
```
# Run, but do not edit, this cell
mod1.mse_resid
```

In [ ]:
```
# Run, but do not edit, this cell
mod2.mse_resid
```

Building a `Pipe`line

Your main task of this stage of the project will be to come up with a model that is as predictive as possible. Ultimately, you will be (1) fitting/training your model on data you have access to (the data in `training_data`) and (2) testing it on data you do not have access to (which will evaluate if your model is actually representatitive of reality or if it is overfit to training data).

To get to that point, lets formalize some of our EDA into a data science pipeline (and yes, we'll be using our old friends `.pipe` and `.assign`). First, lets split your data into a local training and test set. This will allow you to train your models on your local training set, and test them on your local test set, to help you avoid overfitting.

In [ ]:
```
train_split, test_split = train_test_split(training_data, test_size=0.2, random_sta
print("Local Training Data Size: ", train_split.shape[0])
print("Local Test Data Size: ", test_split.shape[0])
```

Carefully review the next two code cells. The first builds a starter pipeline based on our EDA

- `log_transform` will create our `Log Sale Price` and `Log Building Square Feet` columns
- `add_intercept` adds an intercept column to a data frame
- `select_vars` will be used to select just the explanatory features we want to fit our model on
- `filter_training_data` will be used to filter out any observations that may skew our training data
- `train_and_predict` puts these pieces together into a pipeline that we can run directly on loaded data.
- It uses `filter_training_data` to...filter the training data
- It pipes in `log_transform` and `add_intercept` to add features we've seen may be useful
- It trains a model based on training data and uses that model to predict test data. Note that the model only trains using features in `varlist`

The next code cell runs the pipeline "from start to finish"

- It loads in data directly from the CSV
- It gets the same test/train splits as above
- It filters down the training data using `filter_training_data`
- It uses `train_and_predict` to get predictions on both the test and train set, and then prints out the MSE on both

Make sure you understand the code below, then run both cells to see how they work and see the MSE of one of the first models you explored in phase 1!

In [ ]:
```
# DO NOT CHANGE THIS CELL
def log_transforms(df):
    """
    Input a DataFrame df with columns Sale Price and Building Square Feet
    Returns a new DataFrame that adds the log of each of these
    """
    # Note: This is slightly different formatting for assign than we've seen before
    # It works the exact same, but encasing it in a dictionary allows
    # us to create variable names with spaces in them
    return df.assign(**{
        "Log Sale Price": np.log(df["Sale Price"]),
        "Log Building Square Feet": np.log(df["Building Square Feet"])
    })

def add_intercept(df):
    """
    Input a DataFrame df.
    Returns a new DataFrame with an intercept column
    """
    return df.assign(intercept = 1)


def select_vars(df, varlist):
    """
    Filters down DataFrame df to just the columns in varlist,
    which will be used for prediction
    """
    return df[varlist]

def filter_training_data(df):
    """
    Filters the training data.
    You may add additional conditions here to remove outliers from your training se
    Note: these filters will not be applied to the test set.  Only use this filter
    function if you want to remove outliers from your train set for better model fi
    """
    # Start with the baseline filter from Phase 1
    filtered_df = df[(df['Sale Price'] >= 500) & (df['Sale Price'] <= 50000000)].co

    return filtered_df


def train_and_predict(train_df, test_df):
    """The full pipeline!  Takes in a training set and a test set.
    Applies any feature engineering to your training set and test set
    Trains a model on your updated train_df, using just a selection
    of columns in varlist.  Then evaluates that model on a test set,
    returning predictions for that test set."""
    # Filter Training Data
    train_filtered = filter_training_data(train_df)

    # Feature Engineering
```

```
    train_df = train_filtered.pipe(log_transforms).pipe(add_intercept)
    test_df = test_df.pipe(log_transforms).pipe(add_intercept)

    # Select the variables you would like to use in your model
    varlist = ["intercept", "Log Building Square Feet"]


    # Fit the model on the training data and get predictions for the test set
    X_train = select_vars(train_df, varlist)
    X_test = select_vars(test_df, varlist)
    y_train = train_df["Log Sale Price"]
    model = sm.OLS(y_train, X_train).fit()
    predictions = model.predict(X_test)

    return predictions
```

In [ ]:

```
# DO NOT CHANGE THIS CELL

# Load the training data
with zipfile.ZipFile('cook_county_data.zip') as item:
    with item.open("cook_county_train.csv") as f:
        full_train_df = pd.read_csv(f)

# Split the data into a local training and testing set
# NOTE: this is NOT the same test set that I will be using
train_split, test_split = train_test_split(full_train_df, test_size=0.2, random_sta

# Only use the filters for eval_test_split that I used on the Gradescope test
eval_test_split = test_split[(test_split['Sale Price'] >= 500)
                             & (test_split['Sale Price'] <= 50000000)].copy()

# Filter the train set based on filter_training_data function
eval_train_split = filter_training_data(train_split)

# Generate predictions using the previous cell
test_predictions = train_and_predict(train_split, eval_test_split)
train_predictions = train_and_predict(train_split, eval_train_split)

# Calculate the true Log Sale Price for evaluation
y_true_test = np.log(eval_test_split['Sale Price'])
y_true_train = np.log(eval_train_split['Sale Price'])

# Calculate and print the Mean Squared Error
mse_test = mean_squared_error(y_true_test, test_predictions)
mse_train = mean_squared_error(y_true_train, train_predictions)

print("Local Training Split MSE: ",  mse_train)
print("Local Testing Split MSE: ", mse_test)
```

Question 13a Lets practice working with our pipeline. The two cells below emulate the two ones you just familiarized yourself with, but now have some stuff for you to add!

- Complete the function `add_bedrm` that adds a column `num_bedrm` (as in question 8)

- Complete the function `id_expensive` that adds a column `in_expensive` storing 1.0 if a house is in one of the top 10 most expensive neighborhoods and 0 otherwise (as in question 10)
- Complete the function `expand_wall_material` that should add three dummy columns as in Question 11c
- Incorporate these functions into your pipeline in `filter_training_data`
- In `filter_taining_data`, set

```
varlist = ["intercept", "Log Building Square Feet", "num_bedrm",
                   "in_expensive", "Wall Material Cat_Stucco",
                   "Wall Material Cat_Wood", "Wall Material
Cat_Wood&Masonry"]
```

- Run the two code cells to run your full model, emulating what you did in Question 12b of this phase.

You may add helper functions as you wish, and you should of course add docstrings for methods you add (and comments, where appropriate when following good coding style).

Hint: You've done all of this already. This question is all about adapting your code into a pipeline!

You do not need to edit any markdown cells for this question.

In [ ]:
```
# Do NOT change the first three functions:
def log_transforms(df):
    """
    Input a DataFrame df with columns Sale Price and Building Square Feet
    Returns a new DataFrame that adds the log of each of these
    """
    # Note: This is slightly different formatting for assign than we've seen before
    # It works the exact same, but encasing it in a dictionary allows
    # us to create variable names with spaces in them
    return df.assign(**{
        "Log Sale Price": np.log(df["Sale Price"]),
        "Log Building Square Feet": np.log(df["Building Square Feet"])
    })

def add_intercept(df):
    """
    Input a DataFrame df.
    Returns a new DataFrame with an intercept column
    """
    return df.assign(intercept = 1)


def select_vars(df, varlist):
    """
    Filters down DataFrame df to just the columns in varlist,
    which will be used for prediction
    """
```

```
    return df[varlist]

# You can edit this one if you'd like to change how you filter data
def filter_training_data(df):
    """
    Filters the training data.
    You may add additional conditions here to remove outliers from your training se
    Note: these filters will not be applied to the test set.  Only use this filter
    function if you want to remove outliers from your train set for better model fi
    """
    # Start with the baseline filter from Phase 1
    filtered_df = df[(df['Sale Price'] >= 500) & (df['Sale Price'] <= 5000000)].cop

    return filtered_df

# BEGIN QUESTION 13a

# You will likely want to add helper functions here

def add_bedrm(df):
    ...

def id_expensive(df):
    ...

def expand_wall_material(df):
    ...

# END QUESTION 13a

def train_and_predict(train_df, test_df):
    """The full pipeline!  Takes in a training set and a test set.
    Applies any feature engineering to your training set and test set
    Trains a model on your updated train_df, using just a selection
    of columns in varlist.  Then evaluates that model on a test set,
    returning predictions for that test set."""
    # Filter Training Data
    # You may NOT filter or drop rows from the test data
    # Doing so will break the autograder and may result in a zero
    # However, you may filter additional rows out of the train set
    # if you think they will skew your model fitting

    train_filtered = filter_training_data(train_df)

    # Feature Engineering
    # You may change these lines, piping in additional methods to add features, etc
    train_df = (train_filtered.pipe(log_transforms)
                .pipe(add_intercept)
                .pipe(add_bedrm)
                .pipe(id_expensive)
                .pipe(expand_wall_material))
    test_df =  (test_df.pipe(log_transforms)
                .pipe(add_intercept)
                .pipe(add_bedrm)
                .pipe(id_expensive)
                .pipe(expand_wall_material))
```

```
    # Select the variables you would like to use in your model
    # You may include new variables you've created through your pipeline
    varlist = ["intercept", "Log Building Square Feet", "num_bedrm",
                "in_expensive", "Wall Material Cat_Stucco",
                "Wall Material Cat_Wood", "Wall Material Cat_Wood&Masonry"]


    # Do not change the rest
    X_train = select_vars(train_df, varlist)
    X_test = select_vars(test_df, varlist)
    y_train = train_df["Log Sale Price"]
    model = sm.OLS(y_train, X_train).fit()
    predictions = model.predict(X_test)

    return predictions
```

In [ ]:

```
# DO NOT CHANGE THIS CELL

# Load the training data
with zipfile.ZipFile('cook_county_data.zip') as item:
    with item.open("cook_county_train.csv") as f:
        full_train_df = pd.read_csv(f)

# Split the data into a local training and testing set
# NOTE: this is NOT the same test set that I will be using
# I will be testing on data you have never had access to
train_split, test_split = train_test_split(full_train_df, test_size=0.2, random_sta

# Only use the filters for eval_test_split that I used on the Gradescope test
eval_test_split = test_split[(test_split['Sale Price'] >= 500)
                             & (test_split['Sale Price'] <= 50000000)].copy()

# This filters the train set however you want
# (based on your filter_training_data function)
eval_train_split = filter_training_data(train_split)

# Generate predictions using the previous cell
test_predictions = train_and_predict(train_split, eval_test_split)
train_predictions = train_and_predict(train_split, eval_train_split)

# Calculate the true Log Sale Price for evaluation
y_true_test = np.log(eval_test_split['Sale Price'])
y_true_train = np.log(eval_train_split['Sale Price'])

# Calculate and print the Mean Squared Error
mse_test = mean_squared_error(y_true_test, test_predictions)
mse_train = mean_squared_error(y_true_train, train_predictions)

print("Local Training Split MSE: ",  mse_train)
print("Local Testing Split MSE: ", mse_test)
```

Question 13b: Briefly explain why you know the train MSE should be less than 0.6529622297647072 (i.e. in the initial pipeline I provided you that just did a simple linear

regression using `Log Building Square Feet`). Why is the train MSE not exactly the same as what you found after question 12c for mod2?

Edit this cell with your answer to 13b

Your Turn: Feature Selection

The rest of the project only has two questions, but they're big ones. At this point, it's up to you to play around and find/fit the best model you can. You will ultimately evaluate your model (through Gradescope) on a hidden test set. Whoever gets the best MSE in the class will get bonus points, and (a small) part of your grade will be determined by how good of an MSE you find.

Additional EDA

Question 14 First, use the cell(s) below to do additional EDA on at least one quantitative variable and at least one categorical variable that you think may be useful for predicting `Log Sale Price`.

- For your quantitative variable, make a scatterplot of `Log Sale Price` versus your quantitative variable. Is there a strong relationship? Do you notice any non-linear relationship that might suggest an appropriate transformation of the quantitative variable?
- For your categorical variable, consider the number of categories. If it is a small number, make a grouped boxplot. If there's a clear relationship, it may be worth adding that variable in as a predictor!

Use the code cells below to make your graphs and do your exploration. Then use the Markdown cell to briefly summarize your work. You are, of course, welcome to explore more variables than just 2 (and you may not use sale price or any function of it as a predictor!)

In [ ]:
```
# Code for Question 14 Here
# Add code cells as you see fit
```

Edit this cell with your answer to 14

The Final Task: Fit the Best Possible Model!!

Question 15 Come up with the best possible model you can for predicing `Log Sale Price` from the other features (again, you obviously cannot use `Sale Price` or any transformation of it to predict `Log Sale Price`...). Your final model must be a linear regression model with `statsmodels`. Your final model should also be produced in a pipeline, like you worked with in question 13. Specifically:

1. Add and use any code cells you would like for additional exploration. You might, e.g., add cells quickly fitting several models with `statsmodels` without the rigmarole of incorporating them into your pipeline / writing pandas functions for them.

2. Summarize your findings in the Question 15 markdown cell which should, at a minimum, describe the features you considered, the models you considered, and the train/test MSE of each model you considered.

3. Fill in the cell begining with "# Use this cell for your final pipeline" with your complete pipeline for your best/final model. If you run it and the subsequent cell, you should generate two MSE's. You're more than welcome to add methods from Question 13 or any others you see fit!

4. Take your pipeline and model (from the "# Use this cell for your final pipeline" cell below) into the `ExampleFinalModelSubmission.ipynb` Jupyter notebook, following the template and detailed instructions in that notebook (but basically, you should only have to copy and paste parts of your "# Use this cell for your final pipeline" cell into that notebook, making sure that you DO NOT delete the #SUBMISSION CELL tag in the cell in `ExampleFinalModelSubmission.ipynb` or delete imports) . That notebook will contain your final pipeline and you will submit it to Gradescope for evaluation on the hidden test set.

As a few hints:

- The best models will likely start with thoughtful EDA: thinking about what variables to use, how to transform them, etc. They probably will also not have an explosion of features, to avoid issues of overfitting and multicolinearity.
- You might try fitting polynomials (e.g. trying `Building Square Feet`, `Building Square Feet`^{2}, `Building Square Feet`^{3}, `Building Square Feet`^{4} ...), but be very cautious about overfitting. Without good reason, you probably don't want large polynomials (and note that raising big numbers -- like the square feet -- to large powers my cause numerical precision issues that lead to a worse-fitting model).
- You're welcome to use other libraries and tools from `scikit-learn` (e.g. for running forward selection - or try other methods like its backwards selection) for exploration, but these can't be in your final pipeline. You cannot add libraries to your final pipeline outside of the ones loaded in `ExampleFinalModelSubmission.ipynb`
- If Gradescope crashes when you submit, the most likely issues are (1) that your model is taking too much memory or (2) you did not follow the formatting instructions. Make sure that you are not blisfully copying and expanding giant data frames willy-nilly. Then redownload `ExampleFinalModelSubmission.ipynb` and make sure you are following it to a tee: that your first code cell starts with `# SUBMISSION CELL` and that tag appears nowhere else; that all your pipeline code is in that single cell and the next cell runs as-is, that you have not added additional libraries, etc. If those fixes don't work, start from the `ExampleFinalModelSubmission.ipynb` and slowly add features one

at a time, until finding where Gradescope crashes (runs where Gradescope crashes will not count towards your 3 run limit).

Grading Notes

This question is worth 30 points on Project 3, broken down as follows

- 10 points for experimentation and documentation. You should fit at least 5 different models and (across all those models) try at least 5 different features. These should all be documented in the Question 15 Markdown cell
- 10 points for producing a clear pipeline that extends the work we've done so far (including good coding style)
- 10 points for the quality of your final result, as measured by your MSE on your local test set and the hidden test set. **You may submit your a model for evaluation on the hidden test set up to three times.** As some benchmarks, just using a simple linear regression model with `Building Square Feet` (as in `ExampleFinalModelSubmission.ipynb`) will produce an MSE on the hidden test set of about 0.64. Running the pipeline you created in 4a should lower that MSE to about 0.623. To get a nontrivial number of points here, you should should certainly beat those! Quickly asking AI gave me crap that first crashed Gradescope and then after debugging brought the MSE down to 0.58 (remember that a lower MSE is better!). Getting it down to about 0.52 can be done with a handful of well-choosen quantitative variables, and in my experience, human intuition/experimentation rapidly beat AI. Getting it down to about .5 can be done without too many variables or too much feature engineering, using a handful of well-choosen simple categorical and quantitative variables. The best I did while futzing around and creating/testing this project was 0.3343, but that involved getting fairly fancy. **Gradescope will not block you from more than 3 submissions, but if you have more than 3 submissions where Gradescope did not crash, you should expect to not get full points here. Remember the general AI policies**: you CANNOT upload (or copy and paste) parts of this assignment to any LLM. If you want help writing a small, specific function that you come up with (e.g. "what is the syntax for this transformation?"), that's fine. If you'd like to brainstorm some ideas, that's fine too - feel free to share the list of variables with an LLM and ask which it thinks will be useful, e.g.. But even here, at least, my experience is that human intuition is better than AI because no AI has ever seen the exact test data that I am using. Finally, you can ask AI to explain or help you debug errors, but you can't ask it to write/rewrite your code for you.
- You are allowed to filter or drop rows from the training data however you see fit. You are NOT allowed to filter or drop rows from the test data (though I have pre-filtered that data to only include homes whose sale price was at least $500 and at most $50,000,000). You can use any predictors in the data set as explanatory variables, except that you can NOT use `Sale Price` or its transformations (e.g. you can't use `Log Sale Price` to

predict `Log Sale Price`....). For any other predictor, though, you're more than welcome to use it / apply feature engineering to it / etc.

- The team with the best final MSE will get 5 extra credit points on this project. In the event of ties, those points will be split between the teams tying.
- Your submissions to the leaderboard should use an appropriate, unambiguous name. By default, you should use something like "First name, last initial" (but if you are uncomfortable possibly appearing on the leaderboard, contact me and we can work out a pseudonym!).

Caveat Emptor: I'm trying this "hidden test" approach and Gradescope leaderboard for the first time. If it doesn't work as expected, parts of the grading rubric may evolve.

In [ ]:
```
# Question 15
# Use this cell to create your final pipeline.  Change/add ONLY where indicated.

# Do NOT change the first three functions:
def log_transforms(df):
    """
    Input a DataFrame df with columns Sale Price and Building Square Feet
    Returns a new DataFrame that adds the log of each of these
    """
    # Note: This is slightly different formatting for assign
    # It works the exact same, but encasing it in a dictionary allows
    # us to add spaces to new variable names
    return df.assign(**{
        "Log Sale Price": np.log(df["Sale Price"]),
        "Log Building Square Feet": np.log(df["Building Square Feet"])
    })

def add_intercept(df):
    """
    Input a DataFrame df.
    Returns a new DataFrame with an intercept column
    """
    return df.assign(intercept = 1)


def select_vars(df, varlist):
    """
    Filters down DataFrame df to just the columns in varlist,
    which will be used for prediction
    """
    return df[varlist]

# You can edit this one if you'd like to change how you filter data
def filter_training_data(df):
    """
    Filters the training data.
    You may add additional conditions here to remove outliers from your training se
    Note: these filters will not be applied to the test set.  Only use this filter
    function if you want to remove outliers from your train set for better model fi
    """
    # Start with the baseline filter from Phase 1
```

```
    filtered_df = df[(df['Sale Price'] >= 500) & (df['Sale Price'] <= 50000000)].co

    return filtered_df



# Add other helper functions you'd like here



# End of additional helper functions

def train_and_predict(train_df, test_df):
    """The full pipeline!  Takes in a training set and a test set.
    Applies any feature engineering to your training set and test set
    Trains a model on your updated train_df, using just a selection
    of columns in varlist.  Then evaluates that model on a test set,
    returning predictions for that test set."""
    # Filter Training Data
    # You may NOT filter or drop rows from the test data
    # Doing so will break the autograder and may result in a zero
    # However, you may filter additional rows out of the train set
    # if you think they will skew your model fitting

    train_filtered = filter_training_data(train_df)

    # Feature Engineering
    # You may change these lines, piping in additional methods to add features, etc
    train_df = train_filtered.pipe(log_transforms).pipe(add_intercept)
    test_df = test_df.pipe(log_transforms).pipe(add_intercept)

    # Select the variables you would like to use in your model
    # You may include new variables you've created through your pipeline
    varlist = ["intercept", "Log Building Square Feet"]



    # Do not change
    X_train = select_vars(train_df, varlist)
    X_test = select_vars(test_df, varlist)
    y_train = train_df["Log Sale Price"]
    model = sm.OLS(y_train, X_train).fit()
    predictions = model.predict(X_test)

    return predictions
```

In [ ]:

```
# DO NOT CHANGE THIS CELL

# Load the training data
with zipfile.ZipFile('cook_county_data.zip') as item:
    with item.open("cook_county_train.csv") as f:
        full_train_df = pd.read_csv(f)

# Split the data into a local training and testing set
# NOTE: this is NOT the same test set that I will be using
# I will be testing on data you have never had access to
```

```
train_split, test_split = train_test_split(full_train_df, test_size=0.2, random_sta

# Only use the filters for eval_test_split that I used on the Gradescope test
eval_test_split = test_split[(test_split['Sale Price'] >= 500)
                             & (test_split['Sale Price'] <= 50000000)].copy()

# This filters the train set however you want
# (based on your filter_training_data function)
eval_train_split = filter_training_data(train_split)

# Generate predictions using the previous cell
test_predictions = train_and_predict(train_split, eval_test_split)
train_predictions = train_and_predict(train_split, eval_train_split)

# Calculate the true Log Sale Price for evaluation
y_true_test = np.log(eval_test_split['Sale Price'])
y_true_train = np.log(eval_train_split['Sale Price'])

# Calculate and print the Mean Squared Error
mse_test = mean_squared_error(y_true_test, test_predictions)
mse_train = mean_squared_error(y_true_train, train_predictions)

print("Local Training Split MSE: ",  mse_train)
print("Local Testing Split MSE: ", mse_test)
```

Edit this cell with your answer to 15

Phase 2 Submission Instructions

For Phase 2, you will submit to **three** separate Gradescope assignments:

1. Submit `ExampleFinalModelSubmission.ipynb` (updated with your final pipeline) to the `Project 3 Phase 2 - Model Evaluator` assignment.
2. Submit a PDF of this full notebook to the `Project 3 Phase 2 - PDF` assignment
3. Submit a copy of this python notebook to the `Project 3 Phase 2 - Jupyter Notebook` assignment.

As always, for 2 and 3, be sure you follow the normal instructions:

1. Restart the Kernel (e.g. Kernel --> Restart) and run all cells above this one (e.g. Cell --> Run All Above).
2. Make sure that all of the above cells run without error, including the test cells. Be sure that you pass the provided tests.
3. Create a PDF of your notebook. To do so, (1) hit File -> Download as -> HTML (or potentially Save and File -> Export Notebook As -> HTML, depending on your version of Jupyter/OS) (2) open that HTML file in Google Chrome (I drag from downloads into a new Chrome tab), and (3) save that HTML file as a PDF (I type `ctrl+P` to print and then select `Save as PDF` as my destination).
4. Skim the PDF and make sure all your code and written responses are fully visible. If you notice anything (like your code lines are too long and get cut off; your written answers

are cut off), fix those issues and repeat the above process. If I cannot see your work, it will not be graded.

5. Upload that PDF to `Project 3 Phase 2 - PDF` on Gradescope. Tag all pages with the relevant question, and if you worked with a partner, add them to your submission.
6. Upload your Project 3 Jupyter notebook to `Project 3 Phase 2 - Jupyter Notebook` on Gradescope. If you worked with a partner, add them to your submission. Look at the code you uploaded and verify that you submitted a complete Jupyter notebook, with answers to the above questions / cells run. You do not (and should not) zip anything, include data sources, etc -- just upload your .ipynb file.

As always, if you run into any issues, please be sure to send me documentation of your completed work (e.g. a completed Jupyter notebook) before the deadline.

In [ ]: